HOCHSCHULE ESSLINGEN

OBJEKTORIENTIERTE SYSTEME

OPERATOREN

PROF. DR. JÖRG NITZSCHE
HOCHSCHULE ESSLINGEN
JOERG.NITZSCHE@HS-ESSLINGEN.DE

LERNZIEL

- Verstehen wie man Operatoren definieren kann
- Operatoren implementieren können

HOCHSCHULE ESSLINGEN

1 Motivation
2 Überladen von Operatoren
3 Operatoren als Methoden
4 Operatoren als Funktionen
5 Unäre Operatoren
6 Weitere Beispiele

MOTIVATION

- Operatoren sind geläufig sowohl aus dem täglichen Leben als auch für den Programmierer.
- Operatoren funktionieren für die Grunddatentypen. Warum nicht für eigene Klassentypen?

- Beispiel: Klasse **`Fraction`**, Fraction-Objekte **`f_1, f_2`**

```
f_1 + f_2       statt  f_1.add(f_2)
f_1 == f_2      statt  f_1.isEqual(f_2)
cout << f_1     statt  cout << f_1.toString()
```

OPERATOREN IN C++

- C++ stellt eine Reihe von Operatoren zur Verfügung,
  z.B. `. .* :: ?: sizeof # + - * / % ^ & | ~ ! , = > < <= >= ++ -- == != && ||`
- Operatoren haben eine
  - Bindungsrichtung (Assoziativität) (links oder rechts)
    z.B. 1 * 2 * 3 → (1*2)*3 oder 1*(2*3) ?
  - Stellung (Präfix- oder Postfix-Operator):
    z.B. `x = i++; y = ++i;`
  - Prioritätsreihenfolge/Bindungsstärke (Präzedenz)
    z.B. `x = 1 + 2 * 3; y = (1 + 2) * 3;`
  - Wertigkeit (Stelligkeit, Arität)
    z.B. `x = i++; y = 1 + 2;` (unär, binär, ternär, …)

Rang	Bindung	Zweck	Operator
1	L	Bereichsauflösung	::
2	L	Postfix-Inkrement/-Dekrement Funktionsaufruf,Typumwandlung Zugriff auf Element	++ -- Typ() [] . ->
3	R	Präfix-Inkrement/-Dekrement Einstellige Operatoren Typecast, Speicher	++ -- ! ~ - + * & (Typ) new delete
4	L	Elementzeiger	.* ->*
5	L	Pointrechnung	* / %
6	L	Strichrechnung	+ -
7	L	Bitschieben	<< >>
8	L	3-Wege-Vergleich (C++20)	<=>
9 10	L	Vergleiche	< <= >= > == !=
11 12 13	L	bitweise UND XOR ODER	& ^ |
14 15	L	logisch UND ODER	&& ||
16	R	Entscheidungsoperator, Zuweisung mit + - * / % << >> & | ^ als op	? : = op=
17	L	Liste	,

L linksassoziativ: Gruppierung von links nach rechts — a+b+c bedeutet (a+b)+c

R rechtsassoziativ: Gruppierung von rechts nach links — a=b=c bedeutet a=(b=c)

https://namespace-cpp.de/std/doku.php/kennen/operator, October 2021

HOCHSCHULE ESSLINGEN

1 Motivation
2 Überladen von Operatoren
3 Operatoren als Methoden
4 Operatoren als Funktionen
5 Unäre Operatoren
6 Weitere Beispiele

ÜBERLADEN VON OPERATOREN

- Funktionen in C++ können überladen werden, ebenso Methoden oder Konstruktoren von einer Klasse.
- Operatoren verhalten sich wie Funktionen:
  - `x = i++;` entspräche `x = ++(i);` wenn `++` ein Funktionsname wäre.
  - `x = 1 * 2;` entspräche `x = *(1,2);` wenn `*` ein Funktionsname wäre.
- Die Operatorzeichen alleine können aber nicht als Namen benutzt werden. Ausweg: sie erhalten einen Namen wie beispielsweise `operator*` oder `operator++`.
- Operatoren können damit wie Funktionen/Methoden definiert und ggf. auch überladen werden.

EINSCHRÄNKUNGEN BEIM ÜBERLADEN

- Nur in C++ verfügbare Operatoren können überladen werden.
- Folgende Eigenschaften können nicht geändert werden:
  - Bindungsrichtung
  - Prioritätsreihenfolge
  - Wertigkeit
  - Stellung
  - Wirkung auf Grunddatentypen
- Nicht überladen werden können: `.  .*  ::  ?:  sizeof  #`
- Überladen lassen sich z. B.:

```
+ - * / % ^ & | ~ ! , = > < <= >= ++ --
== != && ||
```

PRINZIP DES ÜBERLADENS VON OPERATOREN

- Operatoren werden in Funktions- oder Methodenaufrufe umgesetzt.
- Die Technik der Überladung kann deshalb auch für Operatoren verwendet werden.
- Der Name eines Operators ⊗ ist `operator⊗` .
- Zur Umsetzung gibt es zwei Möglichkeiten:
  - Operator ist Elementfunktion (Methode)
  - Operator ist Freundfunktion bzw. globale Funktion
- Diese zwei Möglichkeiten unterscheiden sich im Wesentlichen dadurch, dass eine Methode einen impliziten Parameter (`this`) hat und eine Funktion nicht.

HOCHSCHULE ESSLINGEN

1 Motivation
2 Überladen von Operatoren
3 Operatoren als Methoden
4 Operatoren als Funktionen
5 Unäre Operatoren
6 Weitere Beispiele

OPERATOR ALS METHODE

- Der Operator (d.h. die Methode) wird auf dem linken Operanden (also dem this-Objekt) aufgerufen.
- Beispiel mit Operator-Schreibweise:

```
Fraction f_1, f_2, f_3; float f = 3.0;
f_3 = f_1 + f_2;
f_3 = f_1 + f;
```

- Gleiches Beispiel mit Methodenaufruf:

```
Fraction f_1, f_2, f_3; float f = 3.0;
f_3 = f_1.operator+(f_2);
f_3 = f_1.operator+(f);
```

- Der erste Operand kann nur ein Objekt der Klasse sein !!!

OPERATOR + ALS METHODE

```
1  class Point
2  {
3      float _x, _y;
4  public:
5      ... // wie gehabt
6      Point operator+(const Point &p_2)
7      {
8          Point p;
9          p._x = _x + p_2._x;
10         p._y = _y + p_2._y;
11         return p;
12     }
13     Point operator+(float f)
14     {
15         Point p;
16         p._x = _x + f;
17         return p;
18     }
19 };
```

- Man kann nicht nur einen Operator+ definieren
- Ein Überladen ist möglich, damit ein Operator auf unterschiedlichen Typen arbeiten kann.

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Articify - QUIZ

```
1  int main()
2  {
3      Point p_1(5.0, 1.0), p_2(10.0, -1.0);
4      p_1.print();
5      p_2.print();
6      Point p_3;
7      p_3.print();
8      p_3 = p_1 + p_2; // p_1.operator+(p_2);
9      p_3.print();
10     p_3 = p_3 + 10.0; // p_3.operator+(10.0);
11     p_3.print();
12     p_3 = 40.0 + p_3; // 40.operator+(p_3);
13     return 0;
14 }
```

- Ist der Programmcode ausführbar?

Articify - QUIZ: AUFLÖSUNG

```
1  int main()
2  {
3      Point p_1(5.0, 1.0), p_2(10.0, -1.0);
4      p_1.print();
5      p_2.print();
6      Point p_3;
7      p_3.print();
8      p_3 = p_1 + p_2; // p_1.operator+(p_2);
9      p_3.print();
10     p_3 = p_3 + 10.0; // p_3.operator+(10.0);
11     p_3.print();
12     p_3 = 40.0 + p_3; // 40.operator+(p_3);
13     return 0;
14 }
```

- 40.0.operator+(p_3) ist kein Methodenaufruf: 40.0 kann nicht als this-Objekt benutzt werden.
- Lösung: Freund-Funktion, um quasi die Kommutativität des Operators + zu gewährleisten:
  - Point operator+ (float a; Point & p);

HOCHSCHULE ESSLINGEN

1 Motivation
2 Überladen von Operatoren
3 Operatoren als Methoden
4 Operatoren als Funktionen
5 Unäre Operatoren
6 Weitere Beispiele

OPERATOR ALS FUNKTION

- Alle Operanden müssen Argumente der Funktion sein.
- Beispiel mit Operator-Schreibweise:

```
int a;
Fraction f_1, f_2;
f_2 = a + f_1;
```

- Gleiches Beispiel mit Funktionsaufruf:

```
int a; Fraction f_1, f_2;
f_2 = operator+(a,f_1);
```

- **a** ist kein Objekt der Klasse **Fraction** und steht links vom **+**-Operator, deshalb ist in diesem Fall der **+**-Operator **nur als Freundfunktion bzw. globale Funktion** möglich.

FREUNDFUNKTIONEN

- Freundfunktionen einer Klasse sind keine Methoden der Klasse. Sie haben aber die gleichen Zugriffsrechte wie solche.
- Eine Klasse erklärt eine Funktion zur Freundfunktion - und nicht umgekehrt.
- Freundfunktionen können im privaten als auch im öffentlichen Teil der Klasse deklariert werden.
- Freundfunktionen werden nicht wie Instanzmethoden auf Objekten aufgerufen. Man muss ihr ein Objekt der Klasse als Parameter übergeben.

BEISPIEL: FREUNDFUNKTION (1)

```
1  class Student
2  {
3    friend void change_grade(Student &s, float g);
4    friend int get_student_count();
5
6    float _grade;
7    static int _count;
8
9  public:
10   Student()
11   {
12     _grade = 1.3;
13     ++_count;
14   }
15   void print()
16   {
17     cout << "Note: " << _grade << endl;
18   }
19 };
```

BEISPIEL: FREUNDFUNKTION (2)

```
1  // Hier kommen die Freundfunktionen:
2  void change_grade(Student &s, float g) { s._grade = g; }
3  int get_student_count() { return Student::_count; }
4
5  int Student::_count = 0;
6  void main()
7  {
8    cout << get_student_count(); // count == 0
9    Student miller;
10   miller.print();              // grade == 1.3
11   change_grade(miller, 1.0);
12   miller.print();              // grade == 1.0
13   cout << get_student_count(); // count == 1
14 }
```

OPERATOR + ALS FREUND

```
1  class Point
2  {
3      float _x, _y;
4
5  public:
6      friend Point operator+(Point, Point);
7      friend Point operator+(Point, float);
8      friend Point operator+(float, Point);
9      Point(float x, float y) : _x(x), _y(y) {}
10     Point() : _x(0), _y(0) {}
11     void print()
12     {
13         cout << "x = " << _x << endl;
14         cout << "y = " << _y << endl;
15     }
16 };
```

- Die Lösung mittels Funktion ist die allgemeinere. Hier im Beispiel werden auch die vorher als Methoden realisierten Operatoren als Funktionen (Freunde) deklariert.

OPERATOR + ALS FUNKTION

```
1  Point operator+ (const Point &p_1,
2                   const Point &p_2){
3    Point p;
4    p.x = p_1._x + p_2._x;
5    p.y = p_1._y + p_2._y;
6    return p;
7  }
8  Point operator+ (const Point &p_1, float a){
9    Point p;
10   p._x = p_1._x + a;
11   return p;
12 }
13 Point operator+ (float a, const Point &p_2){
14   Point p;
15   p._x = a + p_2._x;
16   return p;
17 }
```

- Den letzten Operator könnte man auch unter Zuhilfename des zweiten realisieren:

```
Point operator+ (float a,
const Point & p_2)
{
  return p_2+a;
}
```

- Operatoren können sich also auch gegenseitig aufrufen. Das erspart in manchen Fällen Programmierarbeit.

AUFRUF DER FUNKTION OPERATOR +

```
1  int main() {
2    Point p_1(5.0, 1.0), p_2(10.0, -1.0);
3    p_1.print(); p_2.print();
4    Point p_3;
5    p_3.print();
6    p_3 = p_1 + p_2;      // operator+(p_1, p_2);
7    p_3.print();
8    p_3 = p_3 + 10.0;     // operator+(p_3, 10.0);
9    p_3.print();
10   p_3 = 40.0 + p_3;     // operator+(40.0, p_3);
11   p_3.print();
12   return 0;
13 }
```

- Nun kann auch der letzte Aufruf erfolgen:

```
p_3 = 40.0 + p_3
```

OPERATOR + ALS GLOBALE (NICHT-FREUND) FKT.

```
1  class Point {
2    float _x, _y;
3  public:
4    Point(float x= 0, float y= 0):_x(x), _y(y){}
5    float get_x() const { return _x;}
6    float get_y() const { return _y;}
7    void print(){
8      cout << "x = " << _x << ", y = " << _y
9           << endl;
10   }
11 };
12 Point operator+(float a, const Point &p){
13     return Point(a+p.get_x(), p.get_y());
14 }   // Implementierung mit globaler
15     // (nicht friend) Funktion
```

- Die Klasse muss die entsprechenden get-Methoden zur Verfügung stellen.

HOCHSCHULE ESSLINGEN

1 Motivation
2 Überladen von Operatoren
3 Operatoren als Methoden
4 Operatoren als Funktionen
5 Unäre Operatoren
6 Weitere Beispiele

ÜBERLADEN UNÄRER OPERATOREN

- 1. Möglichkeit (Instanzmethoden):

  Präfix: `Fraction f;` `++f` wird zu `f.operator++()`

  Postfix: `Fraction f;` `f++` wird zu `f.operator++(int)`

- 2. Möglichkeit (Freundfunktionen/globale Funktionen):

  Präfix: `Fraction f;` `++f` wird zu `operator++(f)`

  Postfix: `Fraction f;` `f++` wird zu `operator++(f, int)`

- Zur Unterscheidung besitzen die Postfix-Operatoren einen zusätzlichen int-Parameter.

OPERATOR ++ ALS METHODE

```
1  class Point {
2    float _x, _y;
3
4  public:
5    Point& operator++(){
6      ++_x;
7      ++_y;
8      return *this;
9    }
10
11   Point operator++(int){
12   // Unterscheidung durch den int-Parameter
13     Point p = *this;
14     _x++;
15     _y++;
16     return p;
17   }
18   ...
19 };
```

- Beim Präfix-Operator kommt das modifizierte Objekt zurück, daher ist das Ergebnisse als Referenzen spezifiziert
- Beim Postfix-Operator das Ergebnis eine Kopie des Objekts vor dem Anwenden des Operators.

AUFRUF DES OPERATORS ++

```
1  int main(){
2    Point p_1(5.0, 1.0);
3    ...
4    Point p_3;
5    p_3.print();
6    p_3 = ++p_1;     // wird zu: p_1.operator++();
7    p_3.print();
8    p_1.print();
9    p_3 = p_1++;     // wird zu: p_1.operator++(0);
10   p_3.print();
11   p_1.print();
12   return 0;
13 }
```

- Wenn man den Compiler die Umsetzung machen lässt, ist das unproblematisch.
- Wenn man selber den Funktionsaufruf hinschreibt, muss man daran denken einen Integerwert zu übergeben.

OPERATOR ++ ALS FREUNDFUNKTION

```
1  class Point {
2    float _x, _y;
3  public:
4    ...
5    friend Point& operator++(Point& p);
6    friend Point operator++(Point& p, int);
7  };
8
9  Point& operator++(Point& p){
10     ++p._x;
11     ++p._y;
12     return p;
13 }
14 Point operator++(Point& p, int){
15     Point q = p;
16     p._x++;
17     p._y++;
18     return q;
19 }
```

- Auch hier wird die Unterscheidung durch den zusätzlichen int-Parameter gemacht, der aber keine Bedeutung hat.

HOCHSCHULE ESSLINGEN

1 Motivation
2 Überladen von Operatoren
3 Operatoren als Methoden
4 Operatoren als Funktionen
5 Unäre Operatoren
6 Weitere Beispiele

ÜBERLADEN DES OPERATORS ==

- Im Zusammenhang mit Zuweisung, Kopie und Destruktor muss auch der Gleichheitsoperator betrachtet werden.
- Dieser wird aber vom Compiler **NICHT** zur Verfügung gestellt.
- Der Gleichheitsoperator == muss also für jede Klasse geeignet implementiert werden.
- Falls der Operator== überschrieben wird, sollte auch der Operator != überschrieben werden.
- Der Operator != sollte dann am besten den Operator == nutzen, damit die Logik eingehalten wird.

OPERATOR == ALS METHODE

```
1  class Point {
2    float _x, _y;
3  public:
4    ...
5    bool operator== (const Point& p){
6      if (_x == p._x && _y == p._y) return true;
7      else return false;
8    };
9  };
10
11
12 cout << (p_1==p_2) << endl;
13 cout << (p_1==p_3) << endl;
```

OPERATOR == ALS FREUNDFUNKTION

```
1  class Point {
2    float _x, _y;
3  public:
4    ...
5    friend bool operator== (const Point &,
6                            const Point &);
7  };
8
9  bool operator== (const Point &p_1,
10                  const Point &p_2){
11   if (p_1._x == p_2._x && p_1._y == p_2._y) return true;
12   else return false;
13 }
14
15 cout << (p_1==p_2) << endl;
16 cout << (p_1==p_3) << endl;
```

OPERATOR == ALS GLOBALE FUNKTION

```
1  class Point {
2    float _x, _y;
3  public:   .....
4    float get_x() const { return _x;}
5    float get_y() const { return _y;}
6  };
7
8  bool operator== (const Point &p_1,
9                   const Point &p_2){
10   if (p_1.get_x() == p_2.get_x() &&
11       p_1.get_y() == p_2.get_y()) return true;
12   else return false;
13 }
14
15 cout << (p_1==p_2) << endl;
16 cout << (p_1==p_3) << endl;
```

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Qrticify - QUIZ

- Kann man den << Operator als Methode definieren?

```
1  class Point
2  {
3      float _x, _y;
4  public:
5      ...
6      // operator+
7      Point operator+(const Point &p_2)
8      {
9          Point p;
10         p._x = _x + p_2._x;
11         p._y = _y + p_2._y;
12         return p;
13     }
14     // operator<<
15 };
16
17
18 Point p
19 cout << p;
```

Articify - QUIZ: AUFLÖSUNG

- Kann man den << Operator als Methode definieren?

- Den << Operator kann man nicht als Methode definieren, da das erste Objekt ein ostream-Objekt (hier cout) ist. Man könnte ihn also höchstens in der Klasse ostream als Methode einfügen, die ist aber vordefiniert und wir dürfen sie nicht ändern.

ÜBERLADEN DES OPERATORS <<

```
1  class Point {
2    float _x, _y;
3  public:
4    ...
5    friend ostream& operator<< (ostream&, const Point&);
6  };
7
8  ostream& operator<< (ostream& s, const Point& p){
9    return s << "x = " << p._x << ", y = " << p._y;
10 }
11
12 Point p_1(10.0, 5.0);
13 cout << p_1 << endl; // Ausgabe: x = 10, y = 5
```

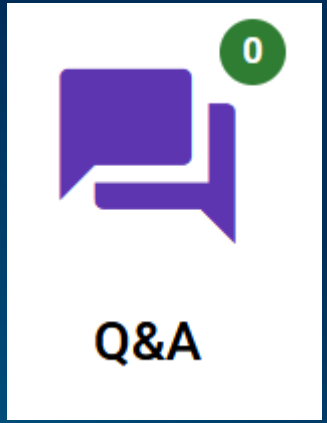
- Die Verkettung ist hier wichtig, damit der neue Operator wie die vordefinierten benutzt werden kann.
- Dafür ist auch wichtig, dass das ostream-Objekt per Referenz ein- und zurückgegeben wird.

ZUSAMMENFASSUNG

- Operatoren haben einen Namen wie beispielsweise `operator*` oder `operator++`.
- Über diesen Namen können Operatoren wie Funktionsaufrufe geschrieben werden.
- Operatoren können damit auch wie Funktionen/Methoden definiert und ggf. überladen werden.
- Wenn ein Operator als Methode realisiert werden soll, muss der erste Operand ein Objekt der Klasse sein können, da der erste Operand zum this-Objekt der Methode wird.
- Wenn ein Operator als Funktion realisiert werden soll, müssen alle Operanden zu Parametern werden. Diese Funktion wird meist als Freundfunktion der Klasse deklariert.

HOCHSCHULE ESSLINGEN

FRAGEN?



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136

HOCHSCHULE ESSLINGEN

KLEINE ÜBUNG

ÜBUNG

Schreiben Sie für das dynamische Array den Operator [], mit dem das Element am angegebenen index zurückgegeben wird.